

动态开点线段树

2026.4.22, 在补25CCPC重庆J, 思路一眼秒了, 但是哥们怎么好像不会写动态开点的线段树?

CF Edu36 E: [Problem - 915E - Codeforces](#)

2024CCPC济南L: [The Tower - Problem - QOJ.ac](#)

2025CCPC重庆J: [Tetris - Problem - QOJ.ac](#)

动态开点线段树

过去的线段树都是处理 $[1, n]$ 的范围, n 一般都是 10^5 的大小, 但是如果要处理 $[1, 10^9]$ 的范围, 甚至是负数的范围, 就需要动态开点的线段树

过去的线段树是直接把长度为 n 的线段树全部创建出来, 而动态开点是, 等我们需要时才创建, 类似于线段树。比如现在有一个节点表示区间为 $[10, 20]$, 需要修改 $[16, 18]$ 的信息时, 就创建表示 $[16, 20]$ 的线段树节点, 然后递归调用, 可能全部操作下来, 都没有创建过表示区间 $[10, 15]$ 的线段树节点

由于空间是动态开的, 所以不会有 $lson = 2 * rt$, $rson = 2 * rt + 1$ 这个性质, 需要记录一下。为了方便调用, 一般会用上宏

```
1  #define ls(x) (tr[x].l)
2  #define rs(x) (tr[x].r)
```

还有需要注意的是, mid 的计算方式变成了 $mid = (l + r - 1) / 2$, 这样可以处理 l, r 都是负数的情况

关于空间复杂度, 用 V 表示值域, 每一次操作最多消耗 $\log(V)$ 的空间, m 次操作为 $m \log(V)$, 一般情况下, 都是在 $[-10^9, 10^9]$ 的范围上操作, $\log(V) = 30$ 。不过空间复杂度一般都是 $O(\text{能开多大开多大})$

区间加、区间查询

```
1  struct segment_tree {
2      #define ls(x) (tr[x].lson)
3      #define rs(x) (tr[x].rson)
4      #define len(x) (tr[x].r - tr[x].l + 1)
5
6      struct node {
7          int l, r;
8          int sum, lazy;
9          int lson, rson;
10         node() : l(0), r(0), sum(0), lazy(0), lson(0), rson(0) {}

```

```

11     node(int l, int r) : l(l), r(r), sum(0), lazy(0), lson(0), rson(0) {}
12 };
13 vector<node> tr;
14
15 int NewNode(int l = 0, int r = 0) {
16     tr.emplace_back(l, r);
17     return tr.size() - 1;
18 }
19
20 segment_tree (int l, int r) {
21     tr.clear();
22     tr.reserve(500010);
23     NewNode(l, r);
24 }
25
26 void pushup(int u) {
27     tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
28 }
29
30 void pushdown(int u) {
31     int l = tr[u].l, r = tr[u].r;
32     int mid = (l + r - 1) / 2;
33     if(!ls(u)) ls(u) = NewNode(l, mid);
34     if(!rs(u)) rs(u) = NewNode(mid + 1, r);
35     if(tr[u].lazy) {
36         tr[ls(u)].sum += tr[u].lazy * len(ls(u));
37         tr[rs(u)].sum += tr[u].lazy * len(rs(u));
38         tr[ls(u)].lazy += tr[u].lazy;
39         tr[rs(u)].lazy += tr[u].lazy;
40         tr[u].lazy = 0;
41     }
42 }
43
44 int query(int u, int l, int r) {
45     if(l <= tr[u].l && tr[u].r <= r) return tr[u].sum;
46     if(l > tr[u].r || r < tr[u].l) return 0;
47     pushdown(u);
48     int mid = (tr[u].l + tr[u].r - 1) / 2;
49     return query(ls(u), l, r) + query(rs(u), l, r);
50 }
51
52 void modify(int u, int l, int r, int x) {
53     if(l <= tr[u].l && tr[u].r <= r) {
54         tr[u].sum += len(u) * x;
55         tr[u].lazy += x;
56         return;
57     }

```

```

58         if(l > tr[u].r || r < tr[u].l) return;
59         pushdown(u);
60         int mid = (tr[u].l + tr[u].r - 1) / 2;
61         if(l <= mid) modify(ls(u), l, r, x);
62         if(r > mid) modify(rs(u), l, r, x);
63         pushup(u);
64     }
65 };

```

Physical Education Lessons

题意：从现在到学期结束还有 n 天（编号从 1 到 n ），起初所有都是工作日。接下来依次发出 q 条指令，每个指令把一段区间 $[l, r]$ 全部变成工作日或非工作日。每条指令发出后，计算剩余的工作日天数

数据范围： $1 \leq n \leq 10^9$, $1 \leq q \leq 3 \cdot 10^5$

```

1 namespace segment_tree {
2     #define ls(x) (tr[x].lson)
3     #define rs(x) (tr[x].rson)
4
5     const int N = 15000010;
6     struct node {
7         int sum, lazy;
8         int lson, rson;
9     }tr[N];
10    int tot;
11
12    int NewNode() {
13        tot++;
14        tr[tot] = {0, -1, 0, 0};
15        return tot;
16    }
17
18    void init(int l, int r) {
19        tot = 0;
20        tr[0] = {0, -1, 0, 0};
21    }
22
23    void pushup(int u) {
24        tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
25    }
26
27    void pushdown(int u, int l, int r) {
28        int mid = (l + r - 1) / 2;
29        if(!ls(u)) ls(u) = NewNode();

```

```

30     if(!rs(u)) rs(u) = NewNode();
31     if(tr[u].lazy == 1) {
32         tr[ls(u)].sum = mid - 1 + 1;
33         tr[ls(u)].lazy = 1;
34         tr[rs(u)].sum = r - mid;
35         tr[rs(u)].lazy = 1;
36         tr[u].lazy = -1;
37     }
38     if(tr[u].lazy == 0) {
39         tr[ls(u)].sum = tr[ls(u)].lazy = 0;
40         tr[rs(u)].sum = tr[rs(u)].lazy = 0;
41         tr[u].lazy = -1;
42     }
43 }
44
45 int query(int u, int l, int r, int L, int R) {
46     if(L <= l && r <= R) return tr[u].sum;
47     if(l > R || r < L) return 0;
48     pushdown(u, l, r);
49     int mid = (l + r - 1) / 2;
50     return query(ls(u), l, mid, L, R) + query(rs(u), mid + 1, r, L, R);
51 }
52
53 void modify(int u, int l, int r, int L, int R, int c) {
54     if(L <= l && r <= R) {
55         tr[u].sum = (r - l + 1) * c;
56         tr[u].lazy = c;
57         return;
58     }
59     if(l > R || r < L) return;
60     pushdown(u, l, r);
61     int mid = (l + r - 1) / 2;
62     if(L <= mid) modify(ls(u), l, mid, L, R, c);
63     if(R > mid) modify(rs(u), mid + 1, r, L, R, c);
64     pushup(u);
65 }
66 };
67
68 void solve() {
69     int n, m;
70     cin >> n >> m;
71     segment_tree::init(1, n);
72
73     while(m --) {
74         int op, l, r;
75         cin >> l >> r >> op;
76         if(op == 1) segment_tree::modify(0, 1, n, l, r, 1);

```

```

77         else segment_tree::modify(0, 1, n, 1, r, 0);
78         cout << n - segment_tree::query(0, 1, n, 1, n) << '\n';
79     }
80 }

```

Tetris

题意：游戏中共有 n 种不同的俄罗斯方块，系统会将每 n 个不同的方块打包成一个 Bag。在一个 Bag 内，这 n 种方块一定会全部出现一次且仅出现一次（内部顺序完全随机），不同 Bag 之间的顺序完全独立。小 C 记住了游戏中连续掉落的 m 个方块的种类，构成了一个序列 $f_1, f_2, \dots, f_m$ 。这段记忆是从全局第 $x + 1$ 个方块开始的，序列中为 0 的位置表示忘记了具体的方块种类，可以匹配任何方块。求有多少个 x ($0 \leq x < n$) 满足序列完美符合 n-BAG 规则，输出符合的 x 的数量及其异或和。序列还会进行 q 次修改操作

数据范围： $1 \leq n \leq 10^{18}$, $1 \leq m \leq 5 \times 10^5$, $0 \leq q \leq 5 \times 10^5$, $0 \leq f_i \leq n$

题解：任意两个相同的元素，不能处于同一个 Bag 中，因此合法的 x 是不超过两段连续区间。用 set 维护每种块的下标，线段树维护每个 x 的合法标记数量、合法 x 个数及其异或和。由于 n 为 10^{18} 级别，线段树需要动态开点

```

1  const int N = 500010;
2  typedef pair<int,int> pii;
3
4  int n, m, q, a[N];
5  int tot;
6  map<int,set<int>> s;
7
8  struct Segment_Tree {
9      #define ls(x) (tr[x].lson)
10     #define rs(x) (tr[x].rson)
11
12     int xorsum(int x) {
13         if(x < 0) return 0;
14         int res = 0;
15         for(int i = x; i >= 0; i--) {
16             if(i % 4 == 3) break;
17             res ^= i;
18         }
19         return res;
20     }
21     int xorsum(int l, int r) {
22         return xorsum(r) ^ xorsum(l - 1);
23     }
24
25     struct node {

```

```

26     int l, r;
27     int cnt, val, sum, lazy;
28     int lson, rson;
29     node() : l(0), r(0), cnt(0), val(0), sum(0), lazy(0), lson(0), rson(0)
{}
30 };
31 vector<node> tr;
32
33 int NewNode(int l = 0, int r = 0) {
34     tr.emplace_back();
35     int pos = tr.size() - 1;
36     tr[pos].l = l, tr[pos].r = r, tr[pos].val = xorsum(l, r), tr[pos].sum
= r - l + 1;
37     return pos;
38 }
39
40 void init(int n) {
41     tr.clear();
42     NewNode(0, n - 1);
43 }
44
45 void pushdown(int u) {
46     int l = tr[u].l, r = tr[u].r;
47     int mid = (l + r - 1) / 2;
48     if(!ls(u)) ls(u) = NewNode(l, mid);
49     if(!rs(u)) rs(u) = NewNode(mid + 1, r);
50     if(tr[u].lazy) {
51         tr[ls(u)].cnt += tr[u].lazy;
52         tr[ls(u)].lazy += tr[u].lazy;
53         tr[rs(u)].cnt += tr[u].lazy;
54         tr[rs(u)].lazy += tr[u].lazy;
55         tr[u].lazy = 0;
56     }
57 }
58
59 void pushup(int u) {
60     if(tr[ls(u)].cnt == tr[rs(u)].cnt) {
61         tr[u].cnt = tr[ls(u)].cnt;
62         tr[u].val = tr[ls(u)].val ^ tr[rs(u)].val;
63         tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
64     } else if(tr[ls(u)].cnt > tr[rs(u)].cnt) {
65         tr[u].cnt = tr[ls(u)].cnt;
66         tr[u].val = tr[ls(u)].val;
67         tr[u].sum = tr[ls(u)].sum;
68     } else {
69         tr[u].cnt = tr[rs(u)].cnt;
70         tr[u].val = tr[rs(u)].val;

```

```

71         tr[u].sum = tr[rs(u)].sum;
72     }
73 }
74
75 void modify(int u, int l, int r, int c) {
76     if(l <= tr[u].l && tr[u].r <= r) {
77         tr[u].cnt += c;
78         tr[u].lazy += c;
79         return;
80     }
81     if(l > tr[u].r || r < tr[u].l) return;
82     pushdown(u);
83     int mid = (tr[u].l + tr[u].r - 1) / 2;
84     if(l <= mid) modify(ls(u), l, r, c);
85     if(r > mid) modify(rs(u), l, r, c);
86     pushup(u);
87 }
88
89 void work() {
90     int x, y;
91     if(tr[0].cnt != tot) x = 0, y = 0;
92     else x = tr[0].sum, y = tr[0].val;
93     cout << x << ' ' << y << '\n';
94 }
95 }Tr;
96
97 vector<pii> get(int x, int y) {
98     vector<pii> res;
99     int px = ((n - x) % n + n) % n;
100    int py = ((n - y + 1) % n + n) % n;
101    if(py <= px) res.push_back({py, px});
102    else {
103        res.push_back({py, n - 1});
104        res.push_back({0, px});
105    }
106    return res;
107 }
108
109 void add(int x, int y) {
110     if(x + n <= y) return;
111     auto p = get(x, y);
112     tot ++;
113     for(auto [l, r]: p)
114         Tr.modify(0, l, r, 1);
115 }
116
117 void del(int x, int y) {

```

```

118     if(x + n <= y) return;
119     auto p = get(x, y);
120     tot --;
121     for(auto [l, r]: p)
122         Tr.modify(0, l, r, -1);
123 }
124
125 void ADD(int pos) {
126     int val = a[pos];
127     if(val == 0) return;
128     auto it = s[val].lower_bound(pos);
129     if((it != s[val].end()) && (it != s[val].begin())) {
130         auto L = prev(it), R = it;
131         int vl = *L, vr = *R;
132         del(vl, vr), add(vl, pos), add(pos, vr);
133     } else if(it != s[val].begin()) {
134         auto L = prev(it);
135         int vl = *L;
136         add(vl, pos);
137     } else if(it != s[val].end()) {
138         auto R = it;
139         int vr = *R;
140         add(pos, vr);
141     }
142     s[val].insert(pos);
143 }
144
145 void DEL(int pos) {
146     int val = a[pos];
147     if(val == 0) return;
148     auto it = s[val].lower_bound(pos);
149     if((it != s[val].begin()) && (next(it) != s[val].end())) {
150         auto L = prev(it), R = next(it);
151         int vl = *L, vr = *R;
152         add(vl, vr), del(vl, pos), del(pos, vr);
153     } else if(it != s[val].begin()) {
154         auto L = prev(it);
155         int vl = *L;
156         del(vl, pos);
157     } else if(next(it) != s[val].end()) {
158         auto R = next(it);
159         int vr = *R;
160         del(pos, vr);
161     }
162     s[val].erase(it);
163 }
164

```



```

165 void solve() {
166     cin >> n >> m >> q;
167     Tr.init(n);
168     for(int i = 1; i <= m; i++) {
169         cin >> a[i];
170         if(a[i]) {
171             if(s[a[i]].size()) add(*s[a[i]].rbegin(), i);
172             s[a[i]].insert(i);
173         }
174     }
175     Tr.work();
176     while(q --) {
177         int x, c;
178         cin >> x >> c;
179         DEL(x);
180         a[x] = c;
181         ADD(x);
182         Tr.work();
183     }
184 }

```

The Tower

题意：维护一个顶部弹幕系统，支持三种操作：

- 发若干条弹幕，它们会占据最靠上的若干个空行
- 删除之前某次发送的全部弹幕
- 查询某一行是否有弹幕，如果有的话返回是哪一次发的

数据范围： $1 \leq n \leq 5 \times 10^5$, $1 \leq k, l \leq 10^9$

题解：因为各个用户的弹幕之间没有联系，因此没有必要把所有行都在一起维护，可以每个用户分别维护一个集合。同时，需要一个集合来维护所有的空行。由于发弹幕的时候涉及到询问名次，需要找到最靠前的空行，可以用线段树来维护这些集合，线段树的每个节点记录该区间内包含的有效行数

操作一涉及到线段树分裂，当新用户发送 k 条弹幕时，会占据当前最靠前的 k 个空行。在操作上，只需要把维护空行的线段树的一个前缀分裂出来放到一棵新的线段树中。这颗新树的根节点就代表了这个新用户占据的所有行，同时用一个 map 存储每个根节点对应的发送者编号

操作二涉及到线段树合并，删除的时候，使用线段树合并将对应集合合并到代表空行的集合中。合并后，该用户原来的根节点作废，维护空行的线段树重新接管这些行

操作三，对于每一个线段树上的节点，维护它的直接祖先，查询的时候只需要跳到根节点然后查询即可

时间复杂度： $O(n(\log v + \log n))$

```

1 struct Segment_Tree {
2     #define ls(x) (tr[x].lson)
3     #define rs(x) (tr[x].rson)
4     #define fa(x) (tr[x].parent)
5
6     struct node {
7         int l, r;
8         int sum;
9         int lson, rson, parent;
10        node() : l(0), r(0), sum(0), lson(0), rson(0), parent(0) {}
11        node(int l, int r) : l(l), r(r), sum(r - l + 1), lson(0), rson(0),
parent(0) {}
12    };
13    vector<node> tr;
14
15    int NewNode(int l = 0, int r = 0) {
16        tr.emplace_back(l, r);
17        return tr.size() - 1;
18    }
19
20    Segment_Tree(int n) {
21        tr.emplace_back(); // 1 base
22        int rt = NewNode(1, n);
23    }
24
25    void pushup(int u) {
26        tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
27    }
28
29    void pushdown(int u) {
30        int mid = (tr[u].l + tr[u].r - 1) / 2;
31        int full = tr[u].sum == (tr[u].r - tr[u].l + 1);
32        if(!ls(u)) {
33            ls(u) = NewNode(tr[u].l, mid);
34            fa(ls(u)) = u;
35            if(!full) tr[ls(u)].sum = 0;
36        }
37        if(!rs(u)) {
38            rs(u) = NewNode(mid + 1, tr[u].r);
39            fa(rs(u)) = u;
40            if(!full) tr[rs(u)].sum = 0;
41        }
42    }
43
44    int build(int u, int x) {
45        if(tr[u].l == tr[u].r) return u;
46        pushdown(u);

```

```

47     int mid = (tr[u].l + tr[u].r - 1) / 2;
48     if(x <= mid) return build(ls(u), x);
49     else return build(rs(u), x);
50 }
51
52 // 线段树分裂, 返回 {原树新根, 新树树根}
53 pii split(int u, int cnt) {
54     if(!u || !cnt) return {u, 0};
55     if(tr[u].sum <= cnt) return {0, u};
56     pushdown(u);
57
58     int v = NewNode();
59     if(cnt <= tr[ls(u)].sum) {
60         pii res = split(ls(u), cnt);
61         ls(u) = res.first, ls(v) = res.second;
62         if(ls(u)) fa(ls(u)) = u;
63         if(ls(v)) fa(ls(v)) = v;
64     } else {
65         cnt -= tr[ls(u)].sum;
66         ls(v) = ls(u), ls(u) = 0;
67         if(ls(v)) fa(ls(v)) = v;
68         pii res = split(rs(u), cnt);
69         rs(u) = res.first, rs(v) = res.second;
70         if(rs(u)) fa(rs(u)) = u;
71         if(rs(v)) fa(rs(v)) = v;
72     }
73
74     pushup(u), pushup(v);
75     return {u, v};
76 }
77
78 int merge(int u, int v) {
79     if(!u || !v) return u | v;
80     if(!tr[u].sum) return v;
81     if(!tr[v].sum) return u;
82
83     int mid = (tr[u].l + tr[u].r - 1) / 2;
84     int nx_ls = merge(ls(u), ls(v));
85     ls(u) = nx_ls;
86     if(ls(u)) fa(ls(u)) = u;
87     int nx_rs = merge(rs(u), rs(v));
88     rs(u) = nx_rs;
89     if(rs(u)) fa(rs(u)) = u;
90     pushup(u);
91     return u;
92 }
93

```

```

94     int find_root(int u) {
95         while(fa(u)) u = fa(u);
96         return u;
97     }
98 };
99
100 void solve() {
101     int n;
102     cin >> n;
103     vector<pii> q(n);
104     vector<int> leaves;
105     for(int i = 0; i < n; i++) {
106         cin >> q[i].first >> q[i].second;
107         if(q[i].first == 3)
108             leaves.push_back(q[i].second);
109     }
110     sort(leaves.begin(), leaves.end());
111     leaves.erase(unique(leaves.begin(), leaves.end()), leaves.end());
112
113     const int V = 1000000000; // 值域
114     Segment_Tree Tr(V);
115     int root = 1;
116     map<int,int> leaf_id;
117     for(auto u: leaves)
118         leaf_id[u] = Tr.build(root, u);
119
120     int userid = 0;
121     map<int,int> user_to_root;
122     map<int,int> root_to_user;
123     for(int i = 0; i < n; i++) {
124         int op = q[i].first, x = q[i].second;
125         if(op == 1) {
126             userid++;
127             pii res = Tr.split(root, x);
128             root = res.first;
129             Tr.tr[root].parent = 0;
130             int v = res.second;
131             if(v) {
132                 Tr.tr[v].parent = 0;
133                 user_to_root[userid] = v;
134                 root_to_user[v] = userid;
135             }
136         }
137         if(op == 2) {
138             if(user_to_root.find(x) != user_to_root.end()) {
139                 int r = user_to_root[x];
140                 root_to_user.erase(r);

```

```
141         user_to_root.erase(x);
142         root = Tr.merge(root, r);
143         Tr.tr[root].parent = 0;
144     }
145 }
146 if(op == 3) {
147     int u = leaf_id[x];
148     u = Tr.find_root(u);
149     if(u == root || root_to_user.find(u) == root_to_user.end()) cout
<< "0\n";
150     else cout << root_to_user[u] << '\n';
151 }
152 }
153 }
```